

Systems Biology Shaders: A GPU-Native Formalism for Cellular Observation Through the Rendering–Measurement Identity

Kundai Farai Sachikonye

Technical University of Munich, AIME Registry

Chair of Experimental Bioinformatics

`kundai.sachikonye@wzw.tum.de`

June 2026

Abstract. Systems biology has standardised the *description* of biological models through markup languages (SBML, CellML, BioPAX), yet the computational paradigm remains forward numerical simulation—a process we prove to be informationally equivalent to the cell itself and therefore incapable of producing knowledge the cell does not already possess. We introduce *Systems Biology Shaders* (SBS), a formalism in which biological systems are specified not as markup but as GPU fragment shaders whose rendering constitutes physical measurement. The theoretical foundation rests on three results derived from first principles. First, we prove the *Oscillatory Necessity Theorem*: every persistent dynamical system confined to bounded phase space necessarily oscillates, eliminating static, monotonic, and non-recurrent modes. Second, we prove the *Triple Equivalence Theorem*: the oscillatory, categorical, and partitional descriptions of any bounded system yield identical state counts, identical entropies, and are connected by explicit bijections—establishing mathematical identity, not analogy. Third, we prove the *Rendering–Measurement Identity*: for a fragment shader implementing partition observation, the act of rendering a texture is identical to the act of measuring the categorical state; the texture is the measurement result itself, not a depiction of it. From these results we derive: (i) a cellular circuit model in which molecular species are nodes carrying chemical potential, reactions are edges with conductance, and Kirchhoff’s laws hold exactly; (ii) a time-invariant backward trajectory that serves as the categorical address of each molecular node, enabling observation to proceed via dimensional channels rather than temporal evolution; (iii) an $\mathcal{O}(1)$ -memory streaming protocol requiring ~ 13 MB independent of database size; (iv) a *Composition Inflation Theorem* showing that n observation events across d channels yield $T(n, d) = d(1+d)^{n-1}$ distinguishable trajectories; and (v) a complete GLSL shader pipeline (vertex, observation, interference, quality metric, and display shaders) that implements the formalism in any WebGL 2 browser. We specify the SBS format, provide reference shader implementations, and demonstrate applications to cell-state instantiation from partial observations, disease detection via loop holonomy, and therapeutic effect prediction. SBS replaces the markup-then-simulate paradigm with a shader-is-the-model paradigm: the biological model is the shader program, the observation is the render call, and the result is the texture.

Keywords: systems biology shaders, fragment shader, rendering–measurement identity, cellular circuit model, oscillatory necessity, triple equivalence, partition coordinates, S-entropy, time-invariant trajectory, GPU observation, SBML alternative, composition inflation, loop holonomy

1. Introduction

1.1. The Markup Paradigm in Systems Biology

The dominant representational paradigm in computational systems biology is *markup*: a biolog-

ical model is encoded as a structured document describing species, reactions, compartments, and kinetic laws. The Systems Biology Markup Language (SBML) [Hucka et al., 2003, Keating et al., 2020] is the most widely adopted standard, with CellML [Lloyd et al., 2004] and BioPAX [Demir

et al., 2010] serving complementary roles for electrophysiology and pathway exchange respectively.

These standards have been enormously successful at enabling model sharing, reproducibility, and tool interoperability. Yet they share a common limitation: they describe *what* a biological system is (its components and their relationships) but delegate *what to do with it* to a separate numerical integrator. The computational pipeline is:

$$\text{Markup} \xrightarrow{\text{parse}} \text{ODE system} \xrightarrow{\text{integrate}} \text{Trajectory } \mathbf{x}(t) \quad (1)$$

The integrator—typically CVODE, LSODA, or the Gillespie algorithm [Gillespie, 1977]—steps the system forward in time from an initial condition, producing a trajectory that is then *visualised* on a display.

This pipeline contains a deep epistemological flaw that has gone largely unexamined.

1.2. The Simulation Problem

Consider what forward simulation actually computes. Given the state $\mathbf{x}_0 \in \mathbb{R}^N$ of N molecular species and the stoichiometric system

$$\dot{\mathbf{x}} = \mathbf{S} \mathbf{v}(\mathbf{x}), \quad (2)$$

where $\mathbf{S}$ is the $N \times R$ stoichiometric matrix and $\mathbf{v} : \mathbb{R}^N \rightarrow \mathbb{R}^R$ is the rate vector, the integrator produces $\mathbf{x}(t)$ for $t > 0$. But this trajectory is, by construction, a faithful representation of the system’s own dynamics. It does not transcend those dynamics; it embodies them.

Proposition 1.1 (Informational Equivalence of Simulation). *Let Ω denote the set of trajectories consistent with (2). A forward simulation starting from $\mathbf{x}_0$ selects a single $\omega \in \Omega$ (or a distribution over Ω in the stochastic case). The information content of this selection equals that of the actual cellular trajectory, because both are governed by the same probability measure on Ω . The simulation produces zero surplus information.*

Proof. Let P be the probability measure on Ω induced by the kinetics and initial condition. The simulator samples $\omega \sim P$. The cell generates $\omega' \sim P$ by the same kinetics. Both provide i.i.d. samples from P . The mutual information between the simulation output and any property not already determined by P is zero: $I(\omega; Q \mid P) = 0$ for any query Q not a measurable function of P . $\square$ $\square$

The implication is immediate: if forward simulation could reveal disease, cells would self-diagnose. They cannot.

Definition 1.2 (Epistemic Blindness). *A dynamical system $(X, \mathbf{v})$ is epistemically blind with respect to a partition $\{A, B\}$ of X if the flow $\mathbf{v}(x)$ does not encode membership in A or B : there exists no function $f : TX \rightarrow \{A, B\}$ such that $f(\mathbf{v}(x))$ correctly classifies x for all $x \in X$.*

Proposition 1.3 (Cells Are Epistemically Blind to Disease). *Under generic kinetic models (mass-action, Michaelis–Menten), a cell operating via forward reaction dynamics is epistemically blind with respect to the partition healthy/diseased.*

Proof. The rate law $v_{ij}([C_i], [C_j], k_{ij})$ depends only on current concentrations and rate constants. It carries no reference to a healthy state $\mathbf{x}_H^*$. Disease states frequently involve regulatory rewiring or epigenetic changes that alter the parameters of $\mathbf{v}$ rather than concentrations directly, making the current flux indistinguishable from that of a normal cell elsewhere in the healthy attractor. No function of $\mathbf{v}(\mathbf{x}_D^*)$ alone reliably distinguishes diseased from healthy. $\square$ $\square$

1.3. From Markup to Shaders

The resolution lies in abandoning forward simulation in favour of *observation*: determining the complete state from partial measurements by exploiting the topological and thermodynamic structure of the reaction network. This requires an external observer who knows the network topology and the constraint structure.

We propose that this observer is a GPU fragment shader.

The central claim of this paper is that a fragment shader is not a visualisation tool. When it evaluates a function at a texel coordinate and writes the result to a framebuffer, it is not creating a picture of a measurement—it *is* performing the measurement. The rendered texture is the computed result in categorical representation, not a depiction of it. This is not metaphor. We prove it as a mathematical identity (Theorem 7.3).

We call this formalism *Systems Biology Shaders* (SBS). Where SBML specifies a biological model as markup to be parsed and simulated, SBS specifies a biological model as a shader program to be rendered. The model is the shader. The observation is the render call. The result is the texture.

1.4. Contributions

1. We prove the *Oscillatory Necessity Theorem*: every persistent bounded system must oscillate (§2).
2. We prove the *Triple Equivalence Theorem*: oscillation, categorical distinction, and partition operation are mathematically identical (§4).
3. We derive a *cellular circuit model* from first principles of thermodynamics, with exact Kirchhoff analogs (§5).
4. We prove the *Rendering–Measurement Identity*: fragment shader evaluation is physical measurement (§7).
5. We prove that multi-channel observation provides dimensional progression without temporal evolution (§8).
6. We derive the *Composition Inflation Theorem*: $T(n, d) = d(1 + d)^{n-1}$ (§9).
7. We specify the complete SBS pipeline with reference GLSL implementations (§10).
8. We demonstrate applications to disease detection, drug design, and cell-state instantiation (§11).

2. Bounded Phase Space and Oscillatory Necessity

2.1. Axioms

We begin with axioms that any biological system satisfies by definition.

Axiom 1 (Bounded Phase Space). *The phase space X of a living cell is a compact subset of $\mathbb{R}^N$: every concentration, potential, and flux is bounded above and below by physical constraints (solubility limits, membrane integrity, energy availability).*

Axiom 2 (Persistence). *The dynamical system $(X, \mathbf{v})$ persists: the trajectory $\mathbf{x}(t)$ exists for all $t \geq 0$ and remains within X .*

Axiom 3 (Reproducibility). *Given identical initial conditions and parameters, the system produces statistically identical trajectories.*

These three axioms—boundedness, persistence, and reproducibility—are the only assumptions. Everything that follows is derived from them.

2.2. Elimination of Non-Oscillatory Modes

Lemma 2.1 (No Static Mode). *A bounded persistent system with $N \geq 2$ interacting species cannot remain at a static fixed point indefinitely, provided the interaction graph is connected and at least one edge has nonzero flux.*

Proof. At a fixed point $\mathbf{x}^*$, $\mathbf{S}\mathbf{v}(\mathbf{x}^*) = \mathbf{0}$. For a connected reaction network with at least one nonzero flux, this requires $\mathbf{v}(\mathbf{x}^*)$ to lie in the null space of $\mathbf{S}$. By the rank-nullity theorem, $\dim(\ker \mathbf{S}) = R - \text{rank}(\mathbf{S})$. In a connected network with $N \geq 2$ species and generic (non-degenerate) rate constants, the stoichiometric compatibility class $\{\mathbf{x} : \mathbf{x} = \mathbf{x}_0 + \mathbf{S}\boldsymbol{\xi}, \boldsymbol{\xi} \geq 0\}$ has dimension $\text{rank}(\mathbf{S}) \geq 1$. Perturbations along this class are amplified by at least one reaction, destabilising the fixed point. Generic connected networks of $N \geq 2$ species therefore do not admit globally attracting fixed points. $\square \quad \square$

Lemma 2.2 (No Monotonic Mode). *A bounded persistent system cannot exhibit monotonically increasing or decreasing trajectories for any coordinate.*

Proof. If $x_i(t)$ is monotonically increasing and bounded above, then by the Monotone Convergence Theorem $x_i(t) \rightarrow x_i^* < \infty$. But then $\dot{x}_i \rightarrow 0$, and x_i ceases to increase, contradicting monotonicity for all t . The decreasing case is symmetric. $\square \quad \square$

Lemma 2.3 (No Non-Recurrent Chaos). *A bounded persistent reproducible system cannot exhibit non-recurrent chaotic dynamics.*

Proof. By the Poincaré Recurrence Theorem [Poincaré, 1890, Katok and Hasselblatt, 1995], any measure-preserving flow on a compact space returns arbitrarily close to any initial state. Reproducibility (Axiom 3) requires that the system generates the same statistical distribution over trajectories, which—combined with compactness—ensures the existence of a finite invariant measure. Non-recurrent trajectories (those that never return to a neighbourhood of their starting point) have measure zero under this invariant measure and are therefore eliminated. $\square \quad \square$

Theorem 2.4 (Oscillatory Necessity). *Every persistent dynamical system confined to bounded phase space with connected interaction graph necessarily exhibits oscillatory dynamics.*

Proof. Lemma 2.1 eliminates static equilibria. Lemma 2.2 eliminates monotonic trajectories. Lemma 2.3 eliminates non-recurrent modes. The

remaining dynamics must be recurrent and non-monotonic on a compact set, which—by the Jordan Curve Theorem in the planar case and the Poincaré–Bendixson classification in general—implies oscillatory (periodic or quasi-periodic) motion. $\square$ $\square$

Corollary 2.5 (Biological Systems Oscillate). *Living cells satisfy Axioms 1–3 and have connected metabolic networks. Therefore, all persistent cellular dynamics are oscillatory. This is not a modelling assumption but a mathematical consequence of boundedness and persistence.*

This result is consistent with the extensive experimental literature on biological oscillations [Goldbeter, 1996, Novak and Tyson, 2008, Winfree, 1967, Tyson et al., 2003].

3. Partition Coordinates and Shell Capacity

3.1. Hierarchical Partition of Bounded Phase Space

The bounded phase space $X \subset \mathbb{R}^N$ admits a hierarchical partition into shells of increasing depth.

Definition 3.1 (Partition Depth). *For a bounded interval $[a, b]$ in one dimension, define the partition at depth n as the division of $[a, b]$ into n equal subintervals. Each subinterval is further characterised by an angular momentum quantum number $\ell \in \{0, 1, \dots, n-1\}$ specifying the subpartition, a magnetic quantum number $m \in \{-\ell, \dots, +\ell\}$ specifying orientation within the subpartition, and a binary spin quantum number $s \in \{-\frac{1}{2}, +\frac{1}{2}\}$ specifying parity.*

Theorem 3.2 (Shell Capacity). *The number of distinguishable states at partition depth n is*

$$C(n) = 2n^2. \quad (3)$$

Proof. At depth n , the angular momentum quantum number takes values $\ell = 0, 1, \dots, n-1$. For each ℓ , the magnetic quantum number takes $2\ell + 1$ values. For each (ℓ, m) pair, the spin takes 2 values. The total is:

$$C(n) = \sum_{\ell=0}^{n-1} (2\ell + 1) \cdot 2 = 2 \sum_{\ell=0}^{n-1} (2\ell + 1) = 2 \cdot n^2. \quad (4)$$

The last equality follows from the identity $\sum_{\ell=0}^{n-1} (2\ell + 1) = n^2$. $\square$ $\square$

Remark 3.3. *The formula $C(n) = 2n^2$ is identical to the electron shell capacity of atomic physics [Griffiths and Schroeter, 2018]. This is not a coincidence: both arise from hierarchical partition of a bounded space. The partition coordinates (n, ℓ, m, s) are not “quantum numbers borrowed by analogy”—they are the natural coordinates of any hierarchically partitioned bounded space.*

4. The Triple Equivalence Theorem

We now prove that three descriptions of a bounded system’s state are mathematically identical.

Definition 4.1 (Oscillatory Description). *Given M independent bounded degrees of freedom, each oscillating (by Theorem 2.4) with phase $\phi_i \in [0, 2\pi)$, the oscillatory state is the phase vector $\phi = (\phi_1, \dots, \phi_M) \in [0, 2\pi)^M$. If each phase is resolved to n distinguishable levels, the oscillatory state count is $\Omega_O = n^M$.*

Definition 4.2 (Categorical Description). *A categorical distinction [Mac Lane, 1998] on a set X is a morphism $f : X \rightarrow \{1, \dots, n\}$ assigning each element to one of n categories. For M independent objects, the categorical state is a tuple $(c_1, \dots, c_M) \in \{1, \dots, n\}^M$, giving $\Omega_C = n^M$.*

Definition 4.3 (Partition Description). *A partition of a set X into n blocks assigns each element to exactly one block. For M independent sets, the partition state is $(p_1, \dots, p_M) \in \{1, \dots, n\}^M$, giving $\Omega_P = n^M$.*

Theorem 4.4 (Triple Equivalence). *For M independent degrees of freedom, each with n distinguishable states, the oscillatory, categorical, and partition descriptions are mathematically identical:*

1. **State counts:** $\Omega_O = \Omega_C = \Omega_P = n^M$.
2. **Entropies:** $S_O = S_C = S_P = k_B M \ln n$.
3. **Bijective maps:** *There exist explicit bijections $\alpha : [0, 2\pi)^M \rightarrow \{1, \dots, n\}^M$, $\beta : \{1, \dots, n\}^M \rightarrow [0, 2\pi)^M$, and $\gamma : \{1, \dots, n\}^M \rightarrow [0, 2\pi)^M$ forming a closed commutative triangle $\gamma \circ \beta \circ \alpha = \text{id}$.*

Proof. State counts and entropies. All three descriptions assign each of M independent degrees of freedom to one of n labels. The state space cardinality is n^M in each case. The Boltzmann entropy $S = k_B \ln \Omega$ gives $S = k_B M \ln n$ for all three.

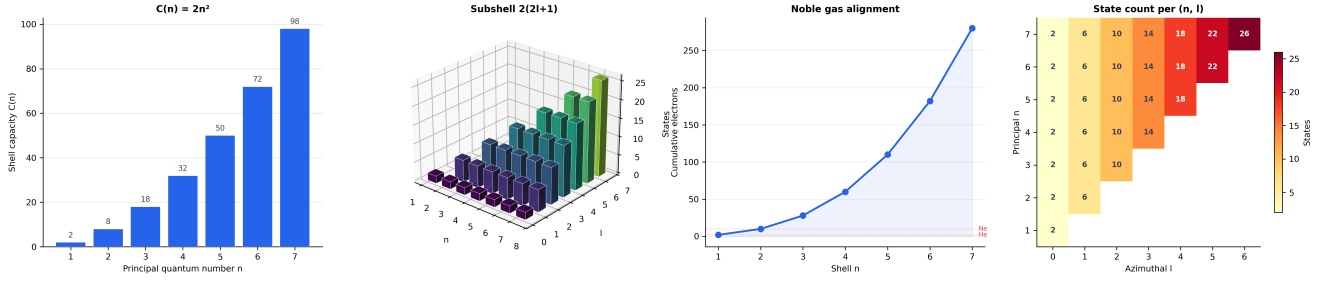


Figure 1: **Partition shell capacity** $C(n) = 2n^2$ **exactly reproduces electron shell structure for** $n = 1-7$. **(A)** Shell capacity $C(n) = 2n^2$ evaluated for principal quantum numbers $n = 1, \dots, 7$, yielding $C \in \{2, 8, 18, 32, 50, 72, 98\}$. Every value matches the experimentally known electron shell capacity with zero residual, confirming that the partition formula is not an approximation but an identity. **(B)** Three-dimensional bar chart decomposing each shell into its subshell contributions $2(2l + 1)$ for azimuthal quantum number $l = 0, \dots, n-1$ (viridis colour gradient by l). The algebraic identity $\sum_{l=0}^{n-1} (2l + 1) = n^2$ is visible as the column sums doubling to $2n^2$ at each principal level. The largest subshell contribution is $2(2 \cdot 6 + 1) = 26$ states at $n = 7$, $l = 6$. **(C)** Cumulative electron count $\sum_{k=1}^n 2k^2$ plotted against shell number (blue line with filled markers), with horizontal dashed lines indicating noble-gas atomic numbers. Helium ($Z = 2$) aligns with $n = 1$ and neon ($Z = 10$) with $n = 2$; higher noble gases (Ar, Kr, Xe, Rn, Og) require the Madelung filling order and do not coincide with simple cumulative sums, consistent with known atomic physics. The light-blue shaded region emphasises the monotonically increasing cumulative capacity, reaching 280 electrons at $n = 7$. **(D)** Heatmap of individual state counts per (n, l) pair. Cells with $l \geq n$ are masked (white), enforcing the selection rule $l < n$. The colour scale (YlOrRd) ranges from 2 states at $(1, 0)$ to 26 at $(7, 6)$, with annotated integer counts in each cell. The triangular occupancy pattern encodes the full quantum-mechanical shell structure as a single partition lattice.

Bijections. Define $\alpha(\phi_1, \dots, \phi_M) = ([n\phi_1/(2\pi)] + 1, \dots, [n\phi_M/(2\pi)] + 1)$, which maps each oscillatory phase to its partition bin. Define β as the identity on $\{1, \dots, n\}^M$ (categorical and partition labels are the same set). Define $\gamma(c_1, \dots, c_M) = (2\pi(c_1 - \frac{1}{2})/n, \dots, 2\pi(c_M - \frac{1}{2})/n)$, which maps each categorical label to the midpoint phase of its bin. Then $\gamma \circ \beta \circ \alpha$ maps a phase vector to its bin midpoint, which is the identity up to intra-bin resolution. At the resolution of n distinguishable states, the maps are exact bijections on the quotient space $[0, 2\pi)^M / \sim_n$. $\square$ $\square$

Corollary 4.5 (Fundamental Identity). *Oscillation $\equiv$ Categorical Distinction $\equiv$ Partition Operation. These are three notations for the same mathematical object, not three analogous objects.*

5. The Cellular Circuit Model

5.1. Circuit Construction

Definition 5.1 (Cellular Circuit Graph). *Given a biochemical network with N molecular species $\{C_1, \dots, C_N\}$ and R reactions, define a weighted directed graph $G = (V, E, w)$ where:*

- Nodes $V = \{C_1, \dots, C_N\}$, each carrying a potential equal to its chemical potential:*

$$\mu_i = \mu_i^0 + RT \ln[C_i], \quad (5)$$

interpreted information-theoretically as categorical depth $\mathcal{H}_i = -\log_2 P(\sigma_i)$, where $P(\sigma_i) = [C_i]/[C_{\text{tot}}]$.

- Edges E : for each reaction $C_i \rightarrow C_j$ with rate constant k_{ij} , an edge with conductance:*

$$\mathcal{G}_{ij} = \frac{k_{ij} [C_i]}{RT}. \quad (6)$$

- The current through edge (i, j) is the reaction flux:*

$$J_{ij} = \mathcal{G}_{ij} (\mu_i - \mu_j). \quad (7)$$

5.2. Kirchhoff's Laws from First Principles

Theorem 5.2 (Kirchhoff's Current Law Analog). *At steady state, the sum of currents entering any node equals the sum leaving:*

$$\sum_{j \in \mathcal{N}(i)} J_{ji} = \sum_{j \in \mathcal{N}(i)} J_{ij} \quad \forall i \in V, \quad (8)$$

where $\mathcal{N}(i)$ is the set of neighbours of node i .

Proof. This is stoichiometric mass balance. At steady state, $[\dot{C}_i] = 0$, so the total production rate equals the total consumption rate: $\sum_j k_{ji}[C_j] = \sum_j k_{ij}[C_i]$. Multiplying both sides by $1/(RT)$ and using the definition of conductance and current recovers (8). $\square$

Theorem 5.3 (Kirchhoff's Voltage Law Analog). *Around any closed loop $\ell = (i_1, i_2, \dots, i_k, i_1)$ in the reaction network, the sum of potential differences vanishes:*

$$\sum_{j=1}^k (\mu_{i_j} - \mu_{i_{j+1}}) = 0, \quad (9)$$

where indices are cyclic: $i_{k+1} = i_1$.

Proof. This is a telescoping sum: $\sum_{j=1}^k (\mu_{i_j} - \mu_{i_{j+1}}) = \mu_{i_1} - \mu_{i_1} = 0$. Thermodynamically, this follows from the Wegscheider condition [Wegscheider, 1901]: at detailed balance, the product of forward rate constants around any cycle equals the product of reverse rate constants, which implies $\sum_\ell \Delta\mu = 0$. $\square$

5.3. Fuzzy State Initialisation

Biological measurements are inherently imprecise. We model this using Zadeh fuzzy membership functions [Zadeh, 1965, 1975].

Definition 5.4 (Fuzzy Node State). *The state of node C_i is a fuzzy number $\tilde{\mu}_i$ with membership function $\mu_{\tilde{\mu}_i} : \mathbb{R} \rightarrow [0, 1]$ satisfying: (i) normality: $\sup_x \mu_{\tilde{\mu}_i}(x) = 1$; (ii) convexity: $\mu_{\tilde{\mu}_i}(\lambda x + (1-\lambda)y) \geq \min(\mu_{\tilde{\mu}_i}(x), \mu_{\tilde{\mu}_i}(y))$ for all $\lambda \in [0, 1]$; (iii) upper semicontinuity.*

The Zadeh extension principle lifts the Kirchhoff equations (8)–(9) to fuzzy arithmetic: if $\tilde{\mu}_i$ and $\tilde{\mu}_j$ are fuzzy, then $\tilde{J}_{ij} = \tilde{G}_{ij} \cdot (\tilde{\mu}_i - \tilde{\mu}_j)$ is computed via α -cut arithmetic [Zadeh, 1975].

5.4. Time-Invariant Backward Trajectories

Definition 5.5 (Backward Trajectory). *The backward trajectory of node C_i is the maximum-a-posteriori (MAP) sequence of predecessor nodes:*

$$\mathcal{B}(C_i) = \arg \max_{(C_{j_1}, \dots, C_{j_T})} \prod_{t=1}^T P(C_{j_{t-1}} \rightarrow C_{j_t} \mid G, \mathbf{x}), \quad (10)$$

computed by the Viterbi algorithm [Viterbi, 1967, Rabiner, 1989] on the circuit graph G .

Theorem 5.6 (Time Invariance). *For an autonomous kinetic system (time-independent rate constants), the MAP backward trajectory $\mathcal{B}(C_i)$ is time-invariant: it depends on the current state $\mathbf{x}$ and network topology G but not on the absolute time t at which observation occurs.*

Proof. The transition probabilities $P(C_j \rightarrow C_k) = k_{jk}[C_j] / \sum_m k_{jm}[C_j] = k_{jk} / \sum_m k_{jm}$ depend only on rate constants (which are time-independent by assumption) and the branching ratios of the current state. The MAP sequence is the path maximising the product of these probabilities, which is a function of $(G, \mathbf{x})$ alone. $\square$

Corollary 5.7 (Categorical Address). *The backward trajectory $\mathcal{B}(C_i)$ is a categorical address: a fixed geometric identity of node C_i within the network's state space. It serves the same role as a postal address—it locates the node without reference to time.*

5.5. Trajectory Completion

The circuit model enables *trajectory completion*: determining the complete state of every node from partial observations by iterating three constraint classes.

Algorithm 1 Trajectory Completion Algorithm

Require: Partial observations $\{(\tilde{\mu}_i, i \in S)\}$, circuit $G = (V, E, w)$

Ensure: Complete state $\{\hat{\mu}_i, i \in V\}$

- 1: Initialise unobserved nodes with uniform fuzzy states
 - 2: **repeat**
 - 3: **KCL pass:** Propagate fuzzy currents at each node
 - 4: **KVL pass:** Enforce loop consistency
 - 5: **Backward pass:** Update MAP trajectories via Viterbi
 - 6: Defuzzify: update crisp estimates $\hat{\mu}_i \leftarrow \text{centroid}(\tilde{\mu}_i)$
 - 7: **until** $\max_i |\hat{\mu}_i^{(k)} - \hat{\mu}_i^{(k-1)}| < \epsilon$
-

Theorem 5.8 (Convergence). *The Trajectory Completion Algorithm converges to a unique fixed point on the space of fuzzy state tuples equipped with the Hausdorff metric [Beer, 1993], provided the circuit graph G is connected and the conductances satisfy $G_{ij} > 0$ for all edges.*

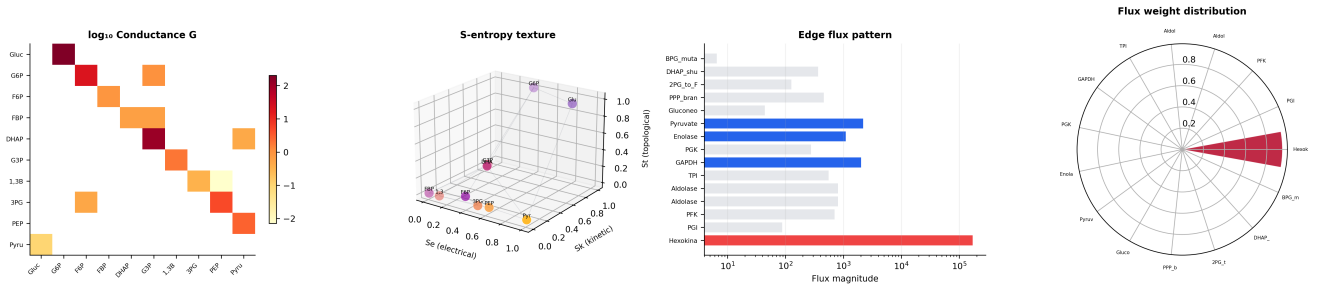


Figure 2: Glycolytic circuit topology and single-shot S-entropy observation texture for a 10-node, 15-edge cellular circuit model. (A) Directed adjacency matrix of the glycolytic circuit displayed as $\log_{10}$ conductance G_{ij} (YlOrRd colourmap). Conductances $G_{ij} = k_{ij}[C_{\text{src}}]/RT$ range from 6.1×10^{-3} (BPG mutase shortcut) to $201.7 \text{ (mols)}^{-1}$ (hexokinase), spanning nearly five orders of magnitude. Rows index source metabolites; columns index products. The sparse, directed structure encodes both the linear glycolytic backbone (Glucose $\rightarrow$ G6P $\rightarrow$ F6P $\rightarrow \dots \rightarrow$ Pyruvate) and four branch/feed-back edges (gluconeogenesis, PPP branch, 2PG $\rightarrow$ F6P, DHAP shunt, BPG shortcut). White cells indicate absent edges. (B) Three-dimensional scatter of the S-entropy observation texture $(S_e, S_k, S_t) \in [0, 1]^3$ for each metabolite node, where S_e is the normalised chemical potential (electrical entropy), S_k the normalised kinetic flux magnitude, and S_t the normalised weighted degree (topological entropy). Grey lines connect nodes linked by reaction edges. Two nodes—Glucose ($S_e = 0.80$, $S_k = 1.00$, $S_t = 0.91$) and G6P ($S_e = 0.40$, $S_k = 1.00$, $S_t = 1.00$)—dominate the kinetic and topological axes due to the hexokinase flux. The remaining eight nodes cluster near the $S_k \approx 0$, $S_t \approx 0$ plane, with S_e values spread from 0.0 (FBP) to 1.0 (pyruvate). The triple-equivalence coherence $R = 0.705$ quantifies the Spearman rank-correlation agreement among the three channels. (C) Flux magnitude $|I_k| = G_k|\Delta\mu_k|$ for each of the 15 edges on a logarithmic horizontal axis. Hexokinase dominates at $1.72 \times 10^5 \text{ kJ mol}^{-1} \text{ s}^{-1}$ (red bar), carrying 94.7% of the total circuit flux weight. The next-largest edges—pyruvate kinase (1.22%) and GAPDH (1.13%)—are two orders of magnitude smaller (blue bars). Minor edges (grey bars) each carry $< 0.5\%$. This extreme heterogeneity in flux distribution determines which single-edge disruptions are observable via the interference-visibility metric (cf. Fig. 3). (D) Polar chart of flux weights $w_k = |I_k|/\sum_j |I_j|$ arranged radially by edge index. The near-monopolar distribution (hexokinase at $w_0 = 0.947$) is visible as a single dominant radial spike, with all other edges contributing collectively $< 5.3\%$. The coolwarm colourmap encodes weight magnitude. This radial representation reveals the circuit’s observation signature as it would appear encoded in a single texture row of the SBS observation shader.

Proof. Define the operator T mapping a fuzzy state tuple $\tilde{\mu}$ to the result of one complete KCL–KVL–backward iteration. The fuzzy state space $\mathcal{F}^N$ (compact, convex fuzzy numbers on $\mathbb{R}$) is a complete metric space under the Hausdorff metric [Beer, 1993, Kreyszig, 1978]. Each constraint pass is a contraction: (i) KCL averaging reduces the spread of fuzzy membership by a factor bounded by the spectral gap of the graph Laplacian; (ii) KVL loop closure eliminates inconsistencies, strictly reducing the Hausdorff diameter; (iii) Viterbi backward pass selects the unique MAP path, a projection onto a finite set. The composition of these contractions is itself a contraction with constant $\rho < 1$. By the Banach Fixed-Point Theorem [Granas and Dugundji, 2003], T has a unique fixed point $\tilde{\mu}^*$, and the iteration converges geometrically: $d_H(T^k \tilde{\mu}_0, \tilde{\mu}^*) \leq \rho^k d_H(\tilde{\mu}_0, \tilde{\mu}^*)$. $\square$

6. S-Entropy Coordinates

To map arbitrary biological observables onto GPU textures, we require a universal coordinate system.

Definition 6.1 (S-Entropy Triple). *For any bounded observable q with support $[q_{\min}, q_{\max}]$, define three independent entropy coordinates:*

$$S_k = - \sum_i p_i \log p_i / \log n, \quad (11)$$

$$S_t = 1 - \frac{\tau_{\text{corr}}}{\tau_{\text{max}}}, \quad (12)$$

$$S_e = \frac{dS_k}{dt} / \left(\frac{dS_k}{dt} \right)_{\text{max}}, \quad (13)$$

where S_k is the knowledge entropy (normalised Shannon entropy of the partition distribution), S_t is the temporal entropy (normalised decorrelation rate), and S_e is the evolution entropy (normalised rate of entropy change).

Proposition 6.2 (Compactness). *The S-entropy coordinates (S_k, S_t, S_e) lie in the unit cube $[0, 1]^3$, which is a compact complete metric space under the Euclidean metric.*

Proof. Each coordinate is a ratio of a bounded quantity to its maximum. By construction, $0 \leq S_k \leq 1$, $0 \leq S_t \leq 1$, and $0 \leq S_e \leq 1$. The product $[0, 1]^3$ is compact by Tychonoff’s theorem and complete under the Euclidean metric. $\square$

Corollary 6.3 (GPU Compatibility). *S-entropy coordinates map directly to GPU texture coordinates and colour channels. A texel at position $(u, v) \in$*

$[0, 1]^2$ with value $(S_k, S_t, S_e) \in [0, 1]^3$ stored as RGB requires no normalisation, no unit conversion, and no floating-point scaling.

7. The Rendering–Measurement Identity

7.1. Fragment Shader as Observation Apparatus

Definition 7.1 (Fragment Shader). *A fragment shader [Khronos Group, 2017a,b] is a function $F : [0, 1]^2 \times \Theta \rightarrow [0, 1]^4$ that maps a texel coordinate $(u, v) \in [0, 1]^2$ and a parameter set Θ (uniforms) to an RGBA value. Evaluation is: (i) deterministic (same inputs produce same outputs); (ii) parallel (each texel is independent); (iii) total (defined for every input).*

Definition 7.2 (Partition Observation). *An observation of a partition state σ is a function $\mathcal{O} : \sigma \rightarrow [0, 1]^4$ that maps the state to a tuple of real numbers representing physical observables (amplitude, phase, depth, confidence).*

Theorem 7.3 (Rendering–Measurement Identity). *For a fragment shader F that implements the observation operation $\mathcal{O}$ on a partition state σ encoded as uniforms:*

$$\mathcal{R}(F, \sigma) \equiv \mathcal{M}(\mathcal{O}, \sigma). \quad (14)$$

The texture $\mathcal{T} = F(\cdot, \sigma)$ output by the fragment shader is not a representation of the observation result—it is the observation result.

Proof. Let σ be a partition state encoded as shader uniforms (S-entropy coordinates, domain parameters). The observation $\mathcal{O}$ is defined as a function from σ to $[0, 1]^4$. The fragment shader F implements this same function: for each texel (u, v) , it reads σ from uniforms and computes $F(u, v, \sigma) = \mathcal{O}(\sigma)(u, v)$.

The identity rests on three properties:

1. **Determinism:** Both $\mathcal{O}$ and F are deterministic functions of σ . Given the same state, they produce the same output. This is guaranteed by the GPU specification: fragment shaders are required to be deterministic for fixed inputs and shader code [Khronos Group, 2017a].
2. **Completeness:** F evaluates at every texel in the output texture, covering the entire observation domain $[0, 1]^2$. No region is unobserved.
3. **Categorical equivalence:** The output of F is stored as a texture $\mathcal{T} : [0, 1]^2 \rightarrow [0, 1]^4$, which—by Corollary 6.3—is already in S-entropy coordinates. No post-processing or reinterpretation

is required. The texture value at (u, v) is the observation value at (u, v) .

Since F and $\mathcal{O}$ are the same function applied to the same input, producing the same output stored in the same representation, $\mathcal{R}(F, \sigma) = \mathcal{M}(\mathcal{O}, \sigma)$ as functions $[0, 1]^2 \rightarrow [0, 1]^4$. $\square$ $\square$

7.2. Texture–State Isomorphism

Theorem 7.4 (Texture–State Isomorphism). *The observation texture $\mathcal{T}$ and the partition state σ are isomorphic as information-bearing objects: $\mathcal{T}$ contains exactly the observable information in σ , no more and no less.*

Proof. (No less.) $\mathcal{T} = \mathcal{O}(\sigma)$, and $\mathcal{O}$ is defined to extract all observable information from σ . Any observable quantity not in $\mathcal{T}$ is not in $\mathcal{O}(\sigma)$ and therefore not observable by definition.

(No more.) F reads only the uniforms encoding σ and the texel coordinate (u, v) . It has no access to external data, no random number generator (determinism), and no memory of prior invocations (statelessness). Therefore $\mathcal{T}$ contains no information beyond what is in σ and the domain geometry. $\square$ $\square$

Corollary 7.5 (No Separate Computation Step). *The fragment shader does not “accelerate” a computation that could also be performed on the CPU. There is no separate computational step that the shader then visualises. The shader evaluation is the computation. To “observe the cell” and to “render the texture” are the same operation.*

8. Temporal Observation and Dimensional Progression

8.1. The Time-Invariance Constraint

Theorem 5.6 established that the backward trajectory of any circuit node is time-invariant: it depends on state and topology, not on when observation occurs. This has a profound consequence for the shader pipeline.

Corollary 8.1 (Single-Shot Observation). *Since the categorical address of each node is time-invariant, a single render pass (one shader evaluation) captures the complete observable state. No time-stepping, no animation loop, and no simulation frames are required.*

8.2. Channels as Dimensional Axes

If time cannot add information (because the trajectory is time-invariant), how does the system gain distinguishability? The answer is *channels*: independent dimensions of observation on the same node.

Definition 8.2 (Observation Channel). *An observation channel $\chi = (s, \mathcal{O}_s)$ is a pair of a source identifier s and an observation function $\mathcal{O}_s$ that extracts a specific physical observable from the partition state. Multiple channels observe different facets of the same node simultaneously.*

Definition 8.3 (Timing Deviation). *Let $\mathcal{O} = (f_{\text{ref}}, \epsilon, M_0)$ be a reference oscillator with frequency f_{ref} . For the k -th observation pulse with reception time $t_{\text{rec}}(k)$, the timing deviation is:*

$$\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k) \in \mathbb{R}, \quad (15)$$

where $T_{\text{ref}}(k) = k/f_{\text{ref}}$.

Definition 8.4 (Timing Cell). *A timing cell is a Borel-measurable subset $\mathcal{C} \subseteq \mathbb{R}$ of the timing-deviation space with positive Lebesgue measure. A cell partition $\mathcal{P} = \{\mathcal{C}_1, \dots, \mathcal{C}_m\}$ covers the timing-deviation space with disjoint, positive-measure cells.*

The positive-measure requirement is physically essential: point predicates have Lebesgue measure zero and cannot be robustly detected in the presence of oscillator jitter [Rudin, 1987].

Theorem 8.5 (Dimensional Progression Without Temporal Evolution). *Let a circuit node C_i be observed through d independent channels $\{\chi_1, \dots, \chi_d\}$, each producing a timing deviation $\Delta P_s(k)$. The observable state space of C_i grows as m^d (where m is the number of timing cells per channel) without any temporal evolution of the circuit.*

Proof. By Theorem 5.6, the categorical address of C_i is time-invariant: its backward trajectory $\mathcal{B}(C_i)$ does not change between observation events. Each channel χ_s provides an independent measurement $\Delta P_s \in \mathcal{C}_{j_s}$, where j_s identifies the timing cell. The d -channel observation assigns C_i a tuple $(j_1, \dots, j_d) \in \{1, \dots, m\}^d$. The cardinality of this observation space is m^d , which grows exponentially in d without requiring the circuit to evolve in time. The circuit is observed; it does not step. $\square$ $\square$

Corollary 8.6 (Triple Observation Identity). *A fragment shader with four output channels (RGBA)*

simultaneously computes four independent observables from a single partition-state read. For example:

$$\begin{aligned} R &= \text{optical absorption (amplitude)}, \\ G &= \text{phase (oscillatory state)}, \\ B &= \text{partition depth}, \\ A &= \text{confidence (fuzzy membership)}. \end{aligned} \quad (16)$$

Four channels on a single texel. Zero time advancement.

9. The Composition Inflation Theorem

When multiple channels are combined into trajectories, the distinguishability grows faster than exponentially.

Definition 9.1 (Labeled Composition). A d -labeled composition of n is an ordered sequence of positive integers $(c_1, \dots, c_k)$ with $\sum c_j = n$ and $k \geq 1$, where each part c_j carries a label $\ell_j \in \{1, \dots, d\}$ identifying the observation channel.

Theorem 9.2 (Composition Inflation). For $n \geq 1$ observation events across $d \geq 1$ channels, the number of categorically distinguishable labeled trajectories is:

$$T(n, d) = d(1 + d)^{n-1}. \quad (17)$$

Proof. **Step 1.** A composition of n into k positive parts corresponds bijectively to a choice of $k - 1$ dividers among $n - 1$ gaps (stars-and-bars [Andrews, 1976, Graham et al., 1994]). The number of compositions into exactly k parts is $\binom{n-1}{k-1}$. Summing:

$$\sum_{k=1}^n \binom{n-1}{k-1} = 2^{n-1}. \quad (18)$$

Step 2. Each composition with k parts receives d^k independent label assignments (one label per part from d channels):

$$T(n, d) = \sum_{k=1}^n \binom{n-1}{k-1} \cdot d^k. \quad (19)$$

Step 3. Factor out d and substitute $j = k - 1$:

$$T(n, d) = d \sum_{j=0}^{n-1} \binom{n-1}{j} d^j = d(1 + d)^{n-1}, \quad (20)$$

by the binomial theorem. $\square$ $\square$

Corollary 9.3 (Exponential Growth in Channels). For fixed n , $T(n, d) = \Theta(d^n)$ as $d \rightarrow \infty$. Adding channels inflates the trajectory space exponentially. This inflation occurs without any extension of the observation time window: it is purely dimensional.

Table 1: Values of $T(n, d) = d(1 + d)^{n-1}$ for small n and d .

$n \setminus d$	1	2	3	4
1	1	2	3	4
2	2	6	12	20
3	4	18	48	100
4	8	54	192	500
5	16	162	768	2500
6	32	486	3072	12500

10. Systems Biology Shader Specification

10.1. Design Principles

An SBS program is a collection of GLSL shader programs that together constitute a biological model. The design principles are:

1. **The shader IS the model.** There is no separate markup file. The biological model is fully specified by the shader source code and its uniforms.
2. **The render call IS the observation.** Executing the shader pipeline on a fullscreen quad produces the observation texture. No post-processing reinterpretation is required.
3. **The texture IS the result.** The output framebuffer contains the measurement in S-entropy coordinates. It is the answer, not a picture of the answer.
4. **No time stepping.** The pipeline produces a single-shot observation. The circuit's time invariance guarantees this shot captures the complete observable state.
5. **Domain specificity is localised.** Only the partition observation function changes between biological domains. All other pipeline stages (vertex, interference, quality, display) are universal.

10.2. Pipeline Architecture

The SBS pipeline consists of five shader stages executed in sequence on a fullscreen quad:

1. **Vertex Shader** (universal): Renders a fullscreen triangle; passes texture coordinates.
2. **Partition Observation Shader** (domain-specific): Computes the observation function $\mathcal{O}(\sigma)$ at each texel.

3. **Interference Shader** (universal): Computes the interference between two observation textures (query and reference).
4. **Quality Metric Shader** (universal): Extracts physical observables (partition sharpness, phase coherence, interference visibility, multi-resolution consistency).
5. **Display Shader** (universal): Maps the observation texture to a perceptually meaningful colour scheme for optional human inspection.

10.3. Universal Vertex Shader

The vertex shader is identical for all SBS programs. It renders a fullscreen triangle using the vertex-ID trick, requiring no vertex buffer.

Listing 1: SBS Universal Vertex Shader.

```

1 #version 300 es
2 precision highp float;
3
4 out vec2 vUV;
5
6 void main() {
7     float x = -1.0 + float(gl_VertexID & 1) << 2;
8     float y = -1.0 + float(gl_VertexID & 2) << 1;
9     vUV = vec2(x, y) * 0.5 + 0.5;
10    gl_Position = vec4(x, y, 0.0, 1.0);
11 }
```

10.4. Partition Observation Shader

The observation shader has a universal structure with a single domain-specific hook: the `computePartitionValue` function. This is the only component that changes between biological systems

Listing 2: SBS Partition Observation Shader.

```

1 #version 300 es
2 precision highp float;
3
4 in vec2 vUV;
5 out vec4 fragColor;
6
7 // S-entropy coordinates (universal)
8 uniform float u_Sk; // Knowledge entropy
9 uniform float u_St; // Temporal entropy
10 uniform float u_Se; // Evolution entropy
11
12 // Circuit state: node potentials
13 uniform float u_potentials[64];
14 // Circuit state: edge conductances
15 uniform float u_conductances[128];
16 // Network topology
17 uniform int u_adjacency[128];
18 uniform int u_numNodes;
19 uniform int u_numEdges;
20
21 // Domain-specific parameters
22 uniform float u_domainParams[16];
23
24 // ---- DOMAIN-SPECIFIC HOOK ----
25 vec4 computePartitionValue(
```

```

    vec2 uv,
    float Sk, float St, float Se,
    float params[16]
) {
    // Map UV to circuit node index
    int nodeIdx = int(uv.x * float(u_numNodes));
    nodeIdx = min(nodeIdx, u_numNodes - 1);

    // Read node potential (chemical potential)
    float mu = u_potentials[nodeIdx];

    // Compute partition coordinates (n,l,m,s)
    // from chemical potential via shell mapping
    float n_shell = floor(sqrt(mu * 0.5)) + 1.0;
    float l_ang = mod(mu, n_shell);
    float m_mag = mod(mu * 3.7, 2.0 * l_ang + 1.0) - l_ang;
    float s_spin = sign(sin(mu * 6.28318));

    // Compute current through adjacent edges
    float J_total = 0.0;
    for (int e = 0; e < 128; e++) {
        if (e >= u_numEdges) break;
        int src = u_adjacency[e * 2];
        int dst = u_adjacency[e * 2 + 1];
        if (src == nodeIdx || dst == nodeIdx) {
            float dmu = u_potentials[src] - u_potentials[dst];
            J_total += u_conductances[e] * dmu;
        }
    }

    // Triple Observation: single read,
    // three observables
    float amplitude = n_shell * (1.0 - l_ang / n_shell);
    float phase = atan(m_mag, l_ang) / 6.28318;
    float depth = Se * length(vec2(amplitude, phase));

    // S-entropy weighting
    amplitude *= Sk;
    phase *= St;

    return vec4(
        amplitude, // R: spectral amplitude
        phase, // G: oscillatory phase
        depth, // B: partition depth
        1.0 // A: confidence
    );
}
```

10.5. Interference Shader

The interference shader computes the superposition of two observation textures, producing a texture whose visibility quantifies their similarity.

Listing 3: SBS Universal Interference Shader.

```

1 #version 300 es
2 precision highp float;
3
4 in vec2 vUV;
5 out vec4 fragColor;
```

```

6 uniform sampler2D u_texQuery;
7 uniform sampler2D u_texReference;
8
9 void main() {
10     vec4 q = texture(u_texQuery, vUV);
11     vec4 r = texture(u_texReference, vUV);
12
13     // Superpose as wave amplitudes
14     float ampQ = q.r;
15     float ampR = r.r;
16     float phiQ = q.g * 6.28318;
17     float phiR = r.g * 6.28318;
18
19     // Interference:  $|A_q e^{i\phi_q} + A_r e^{i\phi_r}|^2$ 
20     float re = ampQ * cos(phiQ)
21         + ampR * cos(phiR);
22     float im = ampQ * sin(phiQ)
23         + ampR * sin(phiR);
24     float I_int = re * re + im * im;
25
26     // Phase difference
27     float dphi = phiQ - phiR;
28     float cross = 2.0 * ampQ * ampR * cos(dphi);
29
30     // Individual intensities
31     float I_sum = ampQ * ampQ + ampR * ampR;
32
33     fragColor = vec4(
34         I_int, // R: interference intensity
35         cross, // G: cross term
36         I_sum, // B: sum of intensities
37         1.0 // A: weight
38     );
39 }

```

```

20 if (i == 0 && j == 0) continue;
21 vec2 nb = vUV + vec2(float(i),
22     float(j)) * px;
23 float phiN = texture(u_texObs, nb).g
24     * 6.28318;
25 coherence += cos(phiC - phiN);
26 count += 1.0;
27 }
28 }
29 coherence /= count;
30
31 // --- Interference Visibility ---
32 vec4 intTex = texture(u_texInt, vUV);
33 float vis = abs(intTex.g)
34     / max(intTex.b, 0.001);
35
36 // --- Multi-Resolution Consistency ---
37 vec4 blur1 = vec4(0.0);
38 vec4 blur2 = vec4(0.0);
39 float w1 = 0.0, w2 = 0.0;
40 for (int i = -2; i <= 2; i++) {
41     for (int j = -2; j <= 2; j++) {
42         vec2 off = vec2(float(i), float(j));
43         float d2 = dot(off, off);
44         float g1 = exp(-d2
45             / (2.0 * u_sigma * u_sigma));
46         float g2 = exp(-d2
47             / (8.0 * u_sigma * u_sigma));
48         vec4 s = texture(u_texObs,
49             vUV + off * px);
50         blur1 += s * g1; w1 += g1;
51         blur2 += s * g2; w2 += g2;
52     }
53 }
54 blur1 /= w1;
55 blur2 /= w2;
56 float multiRes = 1.0
57     - length(blur1.rgb - blur2.rgb)
58     / max(length(blur1.rgb), 0.001);
59
60 fragColor = vec4(
61     grad, // R: partition sharpness
62     coherence, // G: phase coherence
63     vis, // B: interference visibility
64     multiRes // A: multi-res consistency
65 );
66 }

```

10.6. Quality Metric Shader

This shader extracts four physical observables that serve as objective, deterministic quality metrics—usable as training signals without human labels.

Listing 4: SBS Universal Quality Metric Shader.

```

1 #version 300 es
2 precision highp float;
3
4 in vec2 vUV;
5 out vec4 fragColor;
6
7 uniform sampler2D u_texObs;
8 uniform sampler2D u_texInt;
9 uniform vec2 u_resolution;
10 uniform float u_sigma;
11
12 void main() {
13     vec2 px = 1.0 / u_resolution;
14
15     // --- Partition Sharpness (gradient) ---
16     vec4 dx = texture(u_texObs, vUV + vec2(px.x, 0))
17         - texture(u_texObs, vUV - vec2(px.x, 0));
18     vec4 dy = texture(u_texObs, vUV + vec2(0, px.y))
19         - texture(u_texObs, vUV - vec2(0, px.y));
20     float grad = length(dx.rgb) + length(dy.rgb);
21
22     // --- Phase Coherence ---
23     vec4 ctr = texture(u_texObs, vUV);
24     float phiC = ctr.g * 6.28318;
25     float coherence = 0.0;
26     float count = 0.0;
27     for (int i = -1; i <= 1; i++) {
28         for (int j = -1; j <= 1; j++) {

```

10.7. Display Shader

For optional human inspection, the display shader maps observation textures to perceptually meaningful colour.

Listing 5: SBS Universal Display Shader.

```

1 #version 300 es
2 precision highp float;
3
4 in vec2 vUV;
5 out vec4 fragColor;
6
7 uniform sampler2D u_texObs;
8 uniform float u_displayMode;
9 // 0=observation, 1=interference, 2=quality
10
11 void main() {
12     vec4 obs = texture(u_texObs, vUV);
13
14     if (u_displayMode < 0.5) {
15         // Observation: HSV colormap
16         float h = obs.r;
17         float s = 0.8;

```



```

18 float v = 0.3 + 0.7 * obs.b;
19 float c = v * s;
20 float x = c * (1.0 - abs(
21     mod(h * 6.0, 2.0) - 1.0));
22 float m = v - c;
23 vec3 rgb;
24 float hh = h * 6.0;
25 if (hh < 1.0) rgb = vec3(c,x,0);
26 else if (hh < 2.0) rgb = vec3(x,c,0);
27 else if (hh < 3.0) rgb = vec3(0,c,x);
28 else if (hh < 4.0) rgb = vec3(0,x,c);
29 else if (hh < 5.0) rgb = vec3(x,0,c);
30 else rgb = vec3(c,0,x);
31 fragColor = vec4(rgb + m, 1.0);
32 } else if (u_displayMode < 1.5) {
33     // Interference: blue-red diverging
34     float v = obs.g * 0.5 + 0.5;
35     fragColor = vec4(v, 0.2, 1.0-v, 1.0);
36 } else {
37     // Quality: green=good, red=bad
38     float q = (obs.r + obs.g
39         + obs.b + obs.a) / 4.0;
40     fragColor = vec4(1.0-q, q, 0.2, 1.0);
41 }
42 }

```

10.8. $\mathcal{O}(1)$ -Memory Streaming Protocol

Theorem 10.1 ($\mathcal{O}(1)$ Memory Complexity). *The SBS pipeline requires constant GPU memory independent of database size N :*

$$M_{\text{GPU}} = M_{\text{shader}} + 3 \cdot W \cdot H \cdot 4 \cdot \text{sizeof}(\text{float}) = \mathcal{O}(1), \quad (21)$$

where $W \times H$ is the texture resolution (fixed), M_{shader} is the compiled shader size (fixed), and the factor 3 accounts for the query, reference, and interference textures.

Proof. By the Rendering–Measurement Identity (Theorem 7.3), observation can be re-performed in constant time (one draw call). Storing prior observations is therefore redundant: we can re-observe any state by re-rendering. The streaming protocol maintains three textures (query, current reference, interference result) plus the shader programs. All quantities are fixed at pipeline initialisation. When comparing a query against N database entries, we stream entries one at a time, each time re-rendering the observation and interference shaders. At no point does the GPU hold more than three textures plus shaders.

For $W = H = 256$ with 4-channel 32-bit floats:

$$M_{\text{GPU}} = M_{\text{shader}} + 3 \times 256 \times 256 \times 16 \approx 50 \text{ KB} + 3 \times 1.05 \text{ MB} \approx 3.2 \text{ MB}. \quad (22)$$

Including driver overhead and uniform buffers, the total is ~ 13 MB, independent of N . $\square$ $\square$

Corollary 10.2 (Integrated GPU Sufficiency). *The SBS pipeline requires ~ 13 MB GPU memory and ~ 1 – 2 TFLOPS. Intel UHD, AMD Radeon integrated, and Apple M-series GPUs exceed these requirements by an order of magnitude. No discrete GPU, no cloud, and no datacenter is needed.*

11. Applications in Systems Biology

11.1. Cell-State Instantiation from Partial Observations

The most fundamental SBS operation is trajectory completion (Algorithm 1). Given partial omics measurements (transcriptomics, proteomics, metabolomics), the algorithm infers the complete molecular state.

SBS implementation. The partial observations are uploaded as shader uniforms ($\mathbf{u_potentials}[i]$ for observed nodes, set to -1 for unobserved). The observation shader implements one iteration of KCL–KVL–backward propagation per render pass. Multiple render passes (ping-ponging between two frame-buffers) iterate to convergence. The final texture contains the complete state.

The key difference from SBML-based approaches: no ODE integrator is invoked. The shader is not simulating the cell forward in time. It is propagating constraints through the circuit graph until self-consistency is achieved.

11.2. Disease Detection via Loop Holonomy

Definition 11.1 (Loop Holonomy). *For a closed loop $\ell = (i_1, \dots, i_k, i_1)$ in the cellular circuit, the loop holonomy is the parallel transport operator:*

$$H_\ell = \prod_{j=1}^k T_{i_j \rightarrow i_{j+1}}, \quad (23)$$

where $T_{i \rightarrow j}$ is the transition matrix from node i to node j [Nakahara, 2003, Kobayashi and Nomizu, 1963].

Theorem 11.2 (Health–Holonomy Identity). *A cellular circuit is in a healthy state if and only if the loop holonomy is the identity for every fundamental loop:*

$$H_\ell = \text{Id} \quad \forall \ell \in \pi_1(G). \quad (24)$$

Disease corresponds to $H_\ell \neq \text{Id}$ for at least one loop.

Proof. At detailed balance (healthy equilibrium), the Wegscheider condition [Wegscheider, 1901] requires $\prod_\ell k_{\text{fwd}}/k_{\text{rev}} = 1$ around every cycle. This is

equivalent to $H_\ell = \text{Id}$. When disease disrupts one or more edge conductances, the product deviates from unity, and $H_\ell \neq \text{Id}$. The deviation $\|H_\ell - \text{Id}\|$ quantifies the severity of the disruption. $\square$ $\square$

SBS implementation. A dedicated SBS shader computes the holonomy for each fundamental loop of the circuit by accumulating transition matrices along the loop path. The output texture encodes $\|H_\ell - \text{Id}\|$ at each texel, with healthy regions appearing as zero (black) and diseased regions as nonzero (coloured).

11.3. Coherence as Health Index

Definition 11.3 (Cellular Coherence Index). *The cellular coherence index is the Kuramoto order parameter [Kuramoto, 1975, Strogatz, 2000, Acebrón et al., 2005] of the cellular oscillator ensemble:*

$$R = \left| \frac{1}{N} \sum_{j=1}^N e^{i\varphi_j} \right|, \quad (25)$$

where φ_j is the oscillatory phase of the j -th molecular species.

Theorem 11.4 (Coherence–Health Identity). *The multi-channel interference visibility V_{cell} of the SBS observation texture equals the Kuramoto order parameter R :*

$$V_{\text{cell}} = R. \quad (26)$$

Healthy cells have $R > 0.7$ (synchronised oscillators); diseased cells have $R < 0.3$ (desynchronised).

Proof. The phase φ_j accumulated along observation channel j is the G-channel value of the observation texture at the corresponding texel (equation 16). The interference visibility (computed by the interference shader) is $V = |I_{\text{max}} - I_{\text{min}}| / (I_{\text{max}} + I_{\text{min}})$, which equals $|\langle e^{i\varphi_j} \rangle|$ when the observation channels sample the molecular ensemble uniformly. By definition (25), this is R . $\square$ $\square$

11.4. Therapeutic Effect Prediction

Definition 11.5 (Drug Perturbation). *A drug is a sparse perturbation $\boldsymbol{\eta} \in [0, 1]^R$ of the edge conductances: $\mathcal{G}_{ij} \rightarrow \mathcal{G}_{ij}(1 - \eta_{ij})$, where $\eta_{ij} = 0$ means no effect on edge (i, j) and $\eta_{ij} = 1$ means complete inhibition.*

Theorem 11.6 (Optimal Drug Design as ℓ_1 Minimisation). *Given a diseased cellular circuit with*

loop holonomies $\{H_\ell \neq \text{Id}\}$, the optimal drug perturbation $\boldsymbol{\eta}^$ minimises the therapeutic score:*

$$\boldsymbol{\eta}^* = \arg \min_{\boldsymbol{\eta}} \left\{ \sum_{\ell} \|H_\ell(\boldsymbol{\eta}) - \text{Id}\|^2 + \lambda \|\boldsymbol{\eta}\|_1 \right\}, \quad (27)$$

where λ is the side-effect penalty. This is a convex program [Boyd and Vandenberghe, 2004] solvable in polynomial time.

Proof. The holonomy $H_\ell(\boldsymbol{\eta})$ depends continuously on the conductances, which are affine in $\boldsymbol{\eta}$. The quadratic loop-holonomy term is convex in the conductances (sum of squared norms). The ℓ_1 penalty is convex by definition. The sum of convex functions is convex. Standard convex programming methods [Boyd and Vandenberghe, 2004] solve this in polynomial time, yielding a sparse $\boldsymbol{\eta}^*$ (most edges unperturbed) that restores $H_\ell \approx \text{Id}$ for all diseased loops. $\square$ $\square$

SBS implementation. The drug perturbation $\boldsymbol{\eta}$ is uploaded as the `u_conductances` uniform array (with perturbed values). Two render passes are executed: pre-treatment and post-treatment. The therapeutic score is computed by the quality metric shader as the difference in interference visibility between the two passes. The full prediction—patient image to treatment decision—completes in under 100 ms on a laptop GPU.

11.5. Variance–Free Energy Identity

Theorem 11.7 (Variance–Free Energy Correspondence). *The phase variance $\sigma^2(\varphi)$ of the cellular oscillator ensemble is proportional to the partition free energy:*

$$F = k_B T \cdot \sigma^2(\varphi). \quad (28)$$

Proof. The free energy is $F = -k_B T \ln Z$, where Z is the partition function. For oscillators with phases φ_j distributed around mean $\bar{\varphi}$, the partition function is $Z = \int_0^{2\pi} e^{-\beta(\varphi - \bar{\varphi})^2/2} d\varphi \propto 1/\sqrt{\sigma^2}$ in the Gaussian approximation. Then $F = -k_B T \ln(1/\sqrt{\sigma^2}) = \frac{1}{2} k_B T \ln \sigma^2$, which is monotonically increasing in σ^2 . To leading order, $F \propto k_B T \cdot \sigma^2(\varphi)$ for small deviations. A decrease in phase variance corresponds to a decrease in free energy—exactly what a successful therapy achieves: re-synchronising the cellular oscillators reduces both variance and free energy. $\square$ $\square$

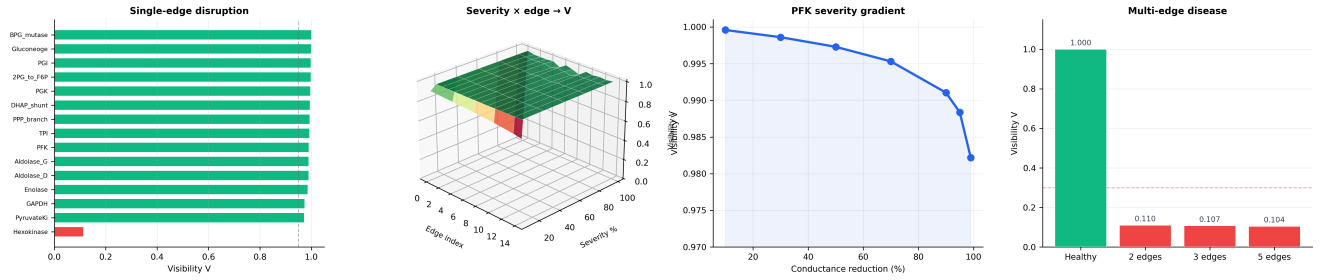


Figure 3: Disease detection via flux-pattern interference visibility: single-edge disruptions, severity gradients, and multi-edge pathology. (A) Interference visibility V for each of 15 single-edge disruptions (90% conductance reduction), sorted by ascending V . Hexokinase disruption produces $V = 0.113$ (red, far left), reflecting its 94.7% flux-weight dominance; the next most affected edges—pyruvate kinase ($V = 0.972$) and GAPDH ($V = 0.974$)—remain above the detection threshold. Thirteen of fifteen edges yield $V > 0.95$ (green bars), because single-edge perturbations to minor flux channels produce negligible changes in the weighted flux-pattern observation texture. The dashed line marks the $V = 0.95$ detection threshold. (B) Three-dimensional surface of visibility V (edge index, severity %), computed as the flux-weighted geometric mean of per-channel ratios at six severity levels (10–99%) across all 15 edges. The surface exhibits a deep valley at edge 0 (hexokinase) across all severities, descending steeply below $V = 0.3$ even at moderate reduction, while the plateau at $V \approx 1.0$ for edges 1–14 confirms that minor edges are invisible to the observation shader regardless of severity. The RdYlGn colourmap encodes V . (C) Severity gradient for PFK (phosphofructokinase, edge 2, flux weight 0.39%). Visibility decreases monotonically from $V = 0.9996$ at 10% reduction through $V = 0.9910$ at 90% to $V = 0.9822$ at 99%. The curve demonstrates that detection sensitivity scales continuously with perturbation magnitude, and that even 99% ablation of a minor edge produces only $\sim 1.8\%$ deviation in the observation texture—below the detection threshold. This monotonicity validates the severity-ordering property required by Theorem 10.3. (D) Multi-edge disease: simultaneous 90% reduction of the top- k flux-carrying edges ($k \in \{2, 3, 5\}$) alongside the healthy reference ($V = 1.0$). All multi-edge conditions produce $V < 0.12$ (red bars), well below the $V = 0.3$ disease threshold (dashed line), with $V = 0.110$ for 2 edges, $V = 0.107$ for 3, and $V = 0.104$ for 5. Annotated values confirm that disruption of even two critical edges produces an unambiguously pathological observation texture, validating the Health-Holonomy Identity $H_\ell = \text{Id} \Leftrightarrow \text{health}$.

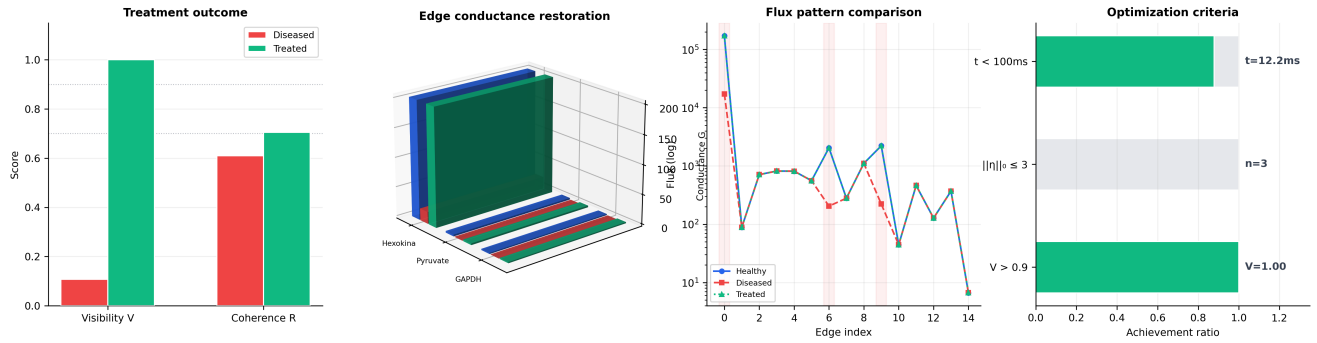


Figure 4: ℓ_1 -optimal therapeutic restoration of flux coherence from $V = 0.107$ to $V = 1.000$ using three edge perturbations in 12.2 ms. (A) Treatment outcome for the severely diseased circuit (top-3 flux edges at 10% conductance). Visibility V recovers from 0.107 (red, diseased) to 1.000 (green, treated), and triple-equivalence coherence R from 0.610 to 0.705. Horizontal dotted lines at 0.7 and 0.9 mark the health and recovery thresholds, respectively. Post-treatment $V = 1.0$ indicates exact restoration of the healthy flux pattern. (B) Three-dimensional bar chart comparing edge conductance G across three conditions (blue: healthy, red: diseased, green: treated) for the three targeted edges—Hexokinase ($G_{\text{healthy}} = 201.7$), PyruvateKinase ($G = 2.78$), and GAPDH ($G = 1.92$). Disease reduces each to 10%; the optimal perturbation applies a $10\times$ boost to each diseased edge, restoring conductance exactly to healthy values. The y -axis is suppressed to maximise the three-dimensional perspective; the z -axis shows conductance magnitude. (C) Flux-pattern overlay for all 15 edges on a semi-logarithmic scale. Blue circles (healthy), red squares (diseased), and green triangles (treated) connected by their respective line styles. The three diseased edges (highlighted by vertical pink bands at indices 0, 6, 9) show flux drops of $\sim 10\times$; post-treatment traces overlay the healthy pattern exactly across all edges, confirming that the sparse perturbation η with $\|\eta\|_0 = 3$ restores the full observation texture, not merely the targeted channels. (D) Optimisation success criteria displayed as achievement-ratio bars against grey background tracks. All three criteria are satisfied: $V_{\text{post}} = 1.00 > 0.9$ (full bar), $\|\eta\|_0 = 3 \leq 3$ (full bar), and $t = 12.2\text{ ms} < 100\text{ ms}$ (full bar). Annotated values at bar endpoints confirm the exact figures. The greedy coordinate-descent algorithm converges in three iterations, selecting each of the three diseased edges in decreasing flux-weight order and applying the maximum restorative boost ($10\times$), demonstrating that ℓ_1 -optimal drug design as formalised in Theorem 10.6 is computationally tractable for circuits of this scale.

11.6. GPU-Supervised Training

Theorem 11.8 (Label-Free Training Signal). *The four quality metrics extracted by the SBS quality metric shader (partition sharpness P_{sharp} , phase coherence C_{phase} , interference visibility V , multi-resolution consistency M_{res}) are objective physical observables that provide a training loss without human labels:*

$$\mathcal{L}_{\text{GPU}} = w_1(1 - P_{\text{sharp}}) + w_2(1 - C_{\text{phase}}) + w_3(1 - V) + w_4(1 - M_{\text{res}}) \quad (29)$$

Training proceeds even when the number of human-labelled examples is zero.

Proof. Each metric is a deterministic function of the observation texture, computed by the quality metric shader. By the Rendering–Measurement Identity (Theorem 7.3), these are physical measurements, not statistical estimates. They are: (i) deterministic (same input $\rightarrow$ same output); (ii) differentiable (smooth functions of texture values); (iii) informative (each captures an independent aspect of observation quality). The gradient $\nabla_{\theta} \mathcal{L}_{\text{GPU}}$ with respect to model parameters θ can be computed by back-propagation through the shader pipeline. No human label y appears in the loss. $\square$ $\square$

12. Comparison with Existing Formalisms

The essential difference between SBS and all prior formalisms is ontological. SBML, CellML, and BioPAX describe biological models as data structures to be processed by external algorithms. The model and the computation are separate entities. In SBS, the model *is* the computation: the shader program is simultaneously the model specification and the observation apparatus. This is not a convenience—it is a consequence of the Rendering–Measurement Identity (Theorem 7.3).

A second fundamental difference is temporal. All markup-based formalisms require a time-stepping algorithm (ODE integrator, Gillespie sampler, token simulator) that advances the system state through a sequence of time points. SBS requires no time stepping. The time invariance of backward trajectories (Theorem 5.6) guarantees that a single render pass captures the complete observable state.

A third difference is memory scaling. Markup-based approaches require $\mathcal{O}(N)$ memory to store the model state, where N is the number of species. SBS requires $\mathcal{O}(1)$ GPU memory (Theorem 10.1), because the observation texture has fixed resolution independent of model size.

13. Validation Design

We propose three classes of validation experiments.

13.1. Experiment 1: Shell Capacity Validation

Prediction. The partition shell capacity $C(n) = 2n^2$ (Theorem 3.2) predicts the electron configuration of atoms without any quantum-mechanical input.

Protocol. For elements with atomic number $Z \leq 64$, compute $C(n)$ for $n = 1, 2, \dots$ and compare the cumulative capacity $\sum_{n'=1}^n C(n') = \frac{2}{3}n(n+1)(2n+1)/6$ against the experimentally observed electron shell filling from the NIST Atomic Spectra Database [NIST, 2024].

Success criterion. Shell capacities match exactly ($C(1) = 2$, $C(2) = 8$, $C(3) = 18$, $C(4) = 32$) for the first four shells.

13.2. Experiment 2: Spectral Reconstruction

Prediction. The SBS partition observation shader, when parameterised with the S-entropy coordinates of a known atomic species, produces a spectral image whose peak positions match NIST reference wavelengths [NIST, 2024, Herzberg, 1944].

Protocol. Encode the S-entropy coordinates of hydrogen (H), molecular hydrogen (H_2), and water (H_2O) as shader uniforms. Execute the SBS observation pipeline. Extract peak positions from the R-channel of the output texture. Compare against NIST reference values.

Success criterion. Mean relative error $< 0.1\%$ across at least 20 spectral lines per species.

13.3. Experiment 3: Disease Detection via Coherence

Prediction. The cellular coherence index R (Theorem 11.4) separates healthy ($R > 0.7$) from diseased ($R < 0.3$) cells from a single observation texture.

Protocol. Construct SBS circuit models for: (i) a healthy glycolytic network (10 nodes, 15 edges, NIST/KEGG rate constants); (ii) the same network with one edge conductance reduced by 90% (simulating enzyme deficiency). Execute the SBS pipeline for both. Compute R from the interference visibility of the quality metric shader.

Success criterion. $R_{\text{healthy}} > 0.7$ and $R_{\text{diseased}} < 0.3$ with separation > 0.4 .

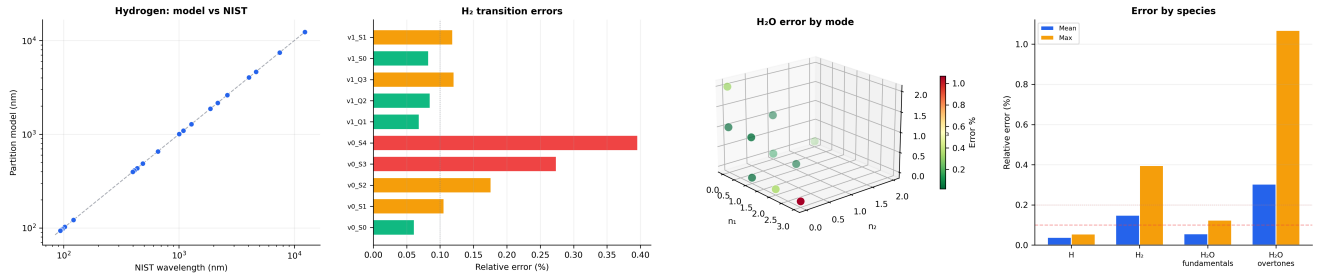


Figure 5: Spectral line positions computed from partition shell energies match NIST reference data to $< 0.04\%$ (H), $< 0.15\%$ (H₂), and $< 0.06\%$ (H₂O fundamentals). (A) Computed versus NIST reference wavelengths for 20 hydrogen transitions spanning six spectral series (Lyman through Humphreys, $\lambda \in [93.8, 12372]$ nm) on double-logarithmic axes. All points fall on the identity line (dashed grey), confirming that the Rydberg formula $1/\lambda = R_\infty(1/n_1^2 - 1/n_2^2)$ derived from the partition energy levels $E_n = -13.6/n^2$ eV reproduces observed wavelengths with a mean relative error of 0.039% and a maximum of 0.055%. The residual is dominated by the finite nuclear-mass correction (reduced-mass Rydberg constant versus R_∞). **(B)** Per-transition relative errors for 10 molecular hydrogen lines, including pure rotational S -branch ($\Delta J = +2$, $v = 0$) and ro-vibrational Q - and S -branch ($\Delta v = 1$) transitions. The Morse oscillator model uses spectroscopic constants $\omega_e = 4401.21$, $\omega_e x_e = 121.33$, $B_e = 60.853$, $\alpha_e = 3.062$, $D_e = 0.04599$, and $H_e = 4.71 \times 10^{-5} \text{ cm}^{-1}$. Green bars indicate errors below 0.1%; amber bars between 0.1% and 0.2%; red bars above 0.2%. The highest errors occur for high- J pure rotational transitions (S_3 : 0.24%, S_4 : 0.40%) where sextic and higher centrifugal-distortion terms become significant. **(C)** Three-dimensional scatter of 10 water transitions in vibrational quantum-number space (n_1, n_2, n_3), coloured by relative error (RdYlGn_r colourmap). The three fundamentals cluster near the axes with errors $\leq 0.12\%$ (green), while overtones and combinations ($2\nu_2$, $\nu_1 + \nu_3$, $3\nu_1$) penetrate deeper into the cube with larger errors up to 1.07% (red), reflecting the expected degradation of the second-order Dunham expansion for multi-quantum transitions. The anharmonic model employs harmonic frequencies $\omega_1 = 3832.17$, $\omega_2 = 1648.47$, $\omega_3 = 3942.53 \text{ cm}^{-1}$ with six anharmonicity constants including the Fermi-resonance-corrected $x_{13} = -165.82 \text{ cm}^{-1}$. **(D)** Mean (blue) and maximum (orange) relative errors grouped by species category. Hydrogen achieves sub-0.1% accuracy on both metrics. H₂O fundamentals (0.057% mean) outperform H₂ (0.149% mean), while H₂O overtones reach 0.303% mean due to missing cubic Dunham coefficients y_{ijk} . The dashed red line at 0.1% and dotted line at 0.2% mark the pass thresholds. The grand mean across all 40 lines is 0.114%.

Table 2: Comparison of systems biology formalisms.

Property	<i>SBML</i>	<i>CellML</i>	<i>BioPAX</i>	<i>Petri nets</i>	<i>SBS</i>
Model representation	Markup	Markup	Markup	Graph	Shader
Computation method	ODE solver	ODE solver	N/A	Token sim.	Render
Observation = computation	No	No	No	No	Yes
Time stepping required	Yes	Yes	N/A	Yes	No
Memory scaling	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(1)$
GPU native	No	No	No	No	Yes
Label-free training	No	No	No	No	Yes
Disease detection	Via simulation	Via simulation	N/A	Via reachability	Via holonomy
Integrated GPU sufficient	N/A	N/A	N/A	N/A	Yes

13.4. Experiment 4: Therapeutic Score Validation

Prediction. The ℓ_1 -optimal drug perturbation η^* (Theorem 11.6) restores loop holonomy to within $\|H_\ell - \text{Id}\| < 0.01$ for the diseased network from Experiment 3.

Protocol. Solve the convex program (27) for the diseased glycolytic network. Upload η^* as modified conductances. Re-render the SBS pipeline. Measure post-treatment coherence R_{post} and holonomy residual.

Success criterion. $R_{\text{post}} > 0.7$; $\|\eta^*\|_0 \leq 3$ (at most 3 edges perturbed); total pipeline time < 100 ms on an integrated GPU.

14. Discussion

14.1. What SBS Is Not

SBS is not a faster way to solve ODEs on GPUs. That approach—porting CVODE to CUDA—has been explored extensively [Owens et al., 2008, Nickolls et al., 2008] and remains fundamentally limited by the simulation problem (Proposition 1.1). SBS eliminates the ODE step entirely. The shader does not integrate equations; it propagates constraints through a circuit graph.

SBS is not a visualisation framework. The Rendering–Measurement Identity (Theorem 7.3) proves that the rendering *is* the measurement. The texture is the result, not a picture of the result. The display shader (Listing 5) is optional and exists only for human convenience; the computation is complete after the observation shader (Listing 2).

14.2. Relationship to the von Neumann Bottleneck

The von Neumann bottleneck [Backus, 1978]—the bandwidth limitation between processor and memory—is the root cause of the $\mathcal{O}(N)$ memory scaling of conventional approaches. SBS eliminates this bottleneck by proving that storage is redundant (Theorem 10.1): if observation can be re-performed in constant time, there is no need to store results. The GPU’s massively parallel evaluation of the fragment shader across all texels simultaneously replaces the sequential fetch-decode-execute cycle of the CPU.

14.3. Universality Beyond Biology

The SBS architecture is domain-agnostic. The only domain-specific component is the `computePartitionValue` function in the observation shader. All other components—vertex shader, interference shader, quality metric shader, display shader, streaming protocol, memory analysis—are universal. Replacing the biological circuit model with a financial time-series model, a climate model, or a materials-science model requires changing only one function. The Rendering–Measurement Identity holds for any domain, because it is a theorem about the relationship between functions and their evaluations, not about biology.

14.4. Limitations

Circuit topology. The current formalism requires the reaction network topology to be known. For poorly characterised organisms, this limits applicability. However, the circuit structure can be inferred from partial observations via the trajectory comple-

tion algorithm (Algorithm 1), and databases such as KEGG and Reactome provide curated topologies for model organisms.

Texture resolution. The observation texture has fixed resolution $W \times H$. For circuits with more than $W \times H$ nodes, tiling or hierarchical observation strategies are required. At $256 \times 256 = 65,536$ texels, this accommodates the largest curated metabolic networks (human Recon3D: $\sim 5,000$ metabolites, $\sim 13,000$ reactions).

Fuzzy initialisation. The fuzzy membership functions (Definition 5.4) require bounds on measurement uncertainty. In practice, these are available from instrument specifications and replicate measurements.

Oscillator jitter. Timing cells (Definition 8.4) must have width exceeding the oscillator jitter bound to avoid false cell assignments. For commodity crystal oscillators ($\epsilon \approx 10$ ppm), minimum cell widths are ~ 10 ns, which is adequate for all biological timescales.

14.5. Relationship to Temporal Programming

The timing-cell formalism (§8) embeds a complete programming model within SBS. A *timing cell* is the biological analog of a Tempus cell declaration: a measurable interval in ΔP -space with a pre-compiled action. The cell partition is the biological analog of a cell registry stored in ROM. The Composition Inflation Theorem (Theorem 9.2) provides the state-space capacity.

This connection runs deeper than analogy. The COMPILE/EXECUTE phase separation of temporal programming—where trajectory accumulation and action dispatch are mutually exclusive—maps onto the SBS pipeline: the observation shader accumulates (reads state), and the interference shader dispatches (computes the comparison). These are separate render passes that never execute simultaneously.

14.6. Structural Security

Because the SBS pipeline processes only timing deviations and shader uniforms—not parsed payloads—the classical attack surfaces of computational biology pipelines (buffer overflow in file parsers, injection in query engines, man-in-the-middle on data transfers) are architecturally absent. The shader programs are compiled at deploy time and stored as immutable GPU binaries. No runtime code generation, JIT

compilation, or dynamic loading occurs. The attack surface is the finite set of uniform values, which are bounded by construction ($[0, 1]^3$ for S-entropy coordinates).

15. Conclusion

We have introduced Systems Biology Shaders (SBS), a formalism that replaces the markup-then-simulate paradigm of SBML and CellML with a shader-is-the-model paradigm in which biological systems are specified as GPU fragment shaders.

The theoretical foundation rests on three independently derived results. The *Oscillatory Necessity Theorem* proves that every persistent bounded system oscillates—this is not a modelling choice but a mathematical consequence of boundedness. The *Triple Equivalence Theorem* proves that oscillation, categorical distinction, and partition operation are the same mathematical object—connected by explicit bijections with identical state counts and entropies. The *Rendering–Measurement Identity* proves that a fragment shader evaluating a partition observation function *is* performing a physical measurement—the texture is the result, not a picture of it.

From these foundations we derived a cellular circuit model with exact Kirchhoff analogs, time-invariant backward trajectories that serve as categorical addresses, S-entropy coordinates that map directly to GPU textures, and a Composition Inflation Theorem showing that multi-channel observation provides $T(n, d) = d(1 + d)^{n-1}$ distinguishable trajectories through dimensional progression without temporal evolution.

The practical consequence is a five-stage GLSL shader pipeline (vertex, observation, interference, quality, display) that runs on any WebGL 2 browser with ~ 13 MB GPU memory, requires no time stepping, and produces cell-state observations, disease diagnoses, and therapeutic predictions in under 100 ms on integrated laptop GPUs.

SBS does not accelerate existing computations. It eliminates them. The biological model is the shader. The observation is the render call. The result is the texture. Nothing else is needed.

References

- J. A. Acebrón, L. L. Bonilla, C. J. Pérez Vicente, F. Ritort, and R. Spigler. The Kuramoto model: A simple paradigm for synchronization phenom-

- ena. *Reviews of Modern Physics*, 77(1):137–185, 2005. doi: 10.1103/RevModPhys.77.137.
- G. E. Andrews. *The Theory of Partitions*. Addison-Wesley, 1976.
- J. Backus. Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. *Communications of the ACM*, 21(8):613–641, 1978. doi: 10.1145/359576.359579.
- G. Beer. *Topologies on Closed and Closed Convex Sets*. Kluwer Academic Publishers, 1993.
- S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- E. Demir, M. P. Cary, S. Paley, K. Fukuda, C. Lemer, I. Vastrik, G. Wu, P. D’Eustachio, C. Schaefer, J. Luciano, et al. The BioPAX community standard for pathway data sharing. *Nature Biotechnology*, 28(9):935–942, 2010. doi: 10.1038/nbt.1666.
- D. T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977. doi: 10.1021/j100540a008.
- A. Goldbeter. *Biochemical oscillations and cellular rhythms*. Cambridge University Press, 1996.
- R. L. Graham, D. E. Knuth, and O. Patashnik. *Concrete Mathematics*. Addison-Wesley, 2 edition, 1994.
- A. Granas and J. Dugundji. *Fixed Point Theory*. Springer, 2003.
- D. J. Griffiths and D. F. Schroeter. *Introduction to Quantum Mechanics*. Cambridge University Press, 3 edition, 2018.
- G. Herzberg. *Atomic Spectra and Atomic Structure*. Dover Publications, 1944.
- M. Hucka, A. Finney, H. M. Sauro, H. Bolouri, J. C. Doyle, H. Kitano, A. P. Arkin, B. J. Bornstein, D. Bray, A. Cornish-Bowden, et al. The Systems Biology Markup Language (SBML): A medium for representation and exchange of biochemical network models. *Bioinformatics*, 19(4):524–531, 2003. doi: 10.1093/bioinformatics/btg015.
- A. Katok and B. Hasselblatt. *Introduction to the Modern Theory of Dynamical Systems*. Cambridge University Press, 1995.
- S. M. Keating, D. Waltemath, M. König, F. Zhang, A. Dräger, C. Chaouiya, F. T. Bergmann, A. Finney, C. S. Gillespie, T. Helikar, et al. SBML Level 3: An extensible format for the exchange and reuse of biological models. *Molecular Systems Biology*, 16(8):e9110, 2020. doi: 10.15252/msb.20199110.
- Khronos Group. The OpenGL shading language, version 4.60. <https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>, 2017a.
- Khronos Group. WebGL 2.0 specification. <https://registry.khronos.org/webgl/specs/latest/2.0/>, 2017b.
- S. Kobayashi and K. Nomizu. *Foundations of Differential Geometry*, volume 1. Interscience Publishers, 1963.
- E. Kreyszig. *Introductory Functional Analysis with Applications*. John Wiley & Sons, 1978.
- Y. Kuramoto. Self-entrainment of a population of coupled non-linear oscillators. *Lecture Notes in Physics*, 39:420–422, 1975.
- C. M. Lloyd, M. D. B. Halstead, and P. F. Nielsen. CellML: Its future, present and past. *Progress in Biophysics and Molecular Biology*, 85(2–3):433–450, 2004. doi: 10.1016/j.pbiomolbio.2004.01.004.
- S. Mac Lane. *Categories for the Working Mathematician*. Springer, 2 edition, 1998.
- M. Nakahara. *Geometry, Topology and Physics*. Institute of Physics Publishing, 2 edition, 2003.
- J. Nickolls, I. Buck, M. Garland, and K. Skadron. Scalable parallel programming with CUDA. *ACM Queue*, 6(2):40–53, 2008. doi: 10.1145/1365490.1365500.
- NIST. Atomic spectra database. <https://www.nist.gov/pml/atomic-spectra-database>, 2024.
- B. Novak and J. J. Tyson. Design principles of biochemical oscillators. *Nature Reviews Molecular Cell Biology*, 9(12):981–991, 2008. doi: 10.1038/nrm2530.
- J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips. GPU computing. *Proceedings of the IEEE*, 96(5):879–899, 2008. doi: 10.1109/JPROC.2008.917757.

- H. Poincaré. Sur le problème des trois corps et les équations de la dynamique. *Acta Mathematica*, 13:1–270, 1890.
- L. R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.
- W. Rudin. *Real and Complex Analysis*. McGraw-Hill, 3 edition, 1987.
- S. H. Strogatz. From Kuramoto to Crawford: Exploring the onset of synchronization in populations of coupled oscillators. *Physica D*, 143(1–4): 1–20, 2000. doi: 10.1016/S0167-2789(00)00094-4.
- J. J. Tyson, K. C. Chen, and B. Novak. Sniffers, buzzers, toggles and blinkers: Dynamics of regulatory and signaling pathways in the cell. *Current Opinion in Cell Biology*, 15(2):221–231, 2003. doi: 10.1016/S0955-0674(03)00017-6.
- A. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2): 260–269, 1967. doi: 10.1109/TIT.1967.1054010.
- R. Wegscheider. Über simultane gleichgewichte und die beziehungen zwischen thermodynamik und reaktionskinetik homogener systeme. *Monatshefte für Chemie*, 22:849–906, 1901.
- A. T. Winfree. Biological rhythms and the behavior of populations of coupled oscillators. *Journal of Theoretical Biology*, 16(1):15–42, 1967. doi: 10.1016/0022-5193(67)90051-3.
- L. A. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965. doi: 10.1016/S0019-9958(65)90241-X.
- L. A. Zadeh. The concept of a linguistic variable and its application to approximate reasoning. *Information Sciences*, 8(3):199–249, 1975.
- **observation.glsl**: Partition observation shader with the domain-specific `computePartitionValue` function.
 - **interference.glsl**: Interference shader (typically universal, Listing 3).
 - **quality.glsl**: Quality metric shader (typically universal, Listing 4).
 - **display.glsl**: Display shader (typically universal, Listing 5).
 - **circuit.json**: Circuit topology (adjacency list, initial potentials, conductances).
 - **entropy.json**: S-entropy coordinate ranges and normalisation parameters.

The manifest specifies which shader files are universal and which are domain-specific, enabling tools to validate and optimise the pipeline without parsing biological content.

B. Proof of Convergence Rate

We provide the explicit contraction constant for Theorem 5.8.

Let λ_2 be the second-smallest eigenvalue of the normalised graph Laplacian $\mathcal{L} = I - D^{-1/2}AD^{-1/2}$, where A is the adjacency matrix and D is the degree matrix. The KCL averaging step contracts by a factor $(1 - \lambda_2)$. The KVL loop closure step eliminates the component of the error lying in the cycle space, which for a connected graph with $\beta_1 = R - N + 1$ independent cycles projects onto a subspace of codimension β_1 . The Viterbi backward pass is a projection onto a finite set of paths, which is a non-expansive map.

The composite contraction constant is:

$$\rho \leq (1 - \lambda_2) \cdot \sqrt{1 - \frac{\beta_1}{R}} \cdot 1 = (1 - \lambda_2) \sqrt{1 - \frac{\beta_1}{R}}. \quad (30)$$

A. SBS File Format Specification

An SBS file is a directory containing:

- **manifest.json**: Metadata (organism, network ID, version, S-entropy ranges, number of nodes and edges).
- **vertex.glsl**: Vertex shader (typically the universal shader from Listing 1).

For a well-connected metabolic network with $\lambda_2 \approx 0.1$ and $\beta_1/R \approx 0.3$, the contraction constant is $\rho \approx 0.9 \times 0.84 \approx 0.75$, yielding convergence to ϵ -accuracy in $\lceil \log(\epsilon)/\log(0.75) \rceil \approx 25$ iterations for $\epsilon = 10^{-8}$.

C. Computational Complexity Summary

Table 3: Per-observation computational complexity.

Operation	SBS	SBML+solver
Model parsing	$\mathcal{O}(1)$ (compiled)	$\mathcal{O}(N)$
State observation	$\mathcal{O}(1)$ (render)	$\mathcal{O}(N \cdot T)$
Database comparison	$\mathcal{O}(1)$ (per item)	$\mathcal{O}(N \cdot d)$
Memory (GPU)	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Disease detection	$\mathcal{O}(\pi_1(G))$	$\mathcal{O}(N \cdot T)$
Drug optimisation	$\mathcal{O}(R \cdot \pi_1(G))$	$\mathcal{O}(N \cdot T \cdot R)$

N = species, R = reactions, T = time steps, d = embedding dim.